

**A DEPTH-FIRST SEARCH BASED APPROACH TO IMPROVING
MOLECULAR PATHWAY SEARCH**

A RESEARCH REPORT SUBMITTED BY
W.M.L.M WIJESUNDARA (S/12/527)

to the

DEPARTMENT OF STATISTICS AND COMPUTER SCIENCE

*In partial fulfilment of the requirement
for the award of the degree of*

BACHELOR OF SCIENCE

of the

**UNIVERSITY OF PERADENIYA
SRI LANKA**

2017

DECLARATION

I do hereby declare that the work reported in this project report/thesis was exclusively carried out by me under the supervision of Dr. R. Siyanbalapitiya and Mr. P.M.P.C. Gunathilaka. It describes the results of my own independent research except where due reference has been made in the text. No part of this project report/thesis has been submitted earlier or concurrently for the same or any other degree.

Date:

.....

Signature of the Candidate

Certified by:

Supervisor: Dr. R Siyanbalapitiya

Date:

(Signature):

Co-Supervisor: Mr. P.M.P.C. Gunathilaka

Date:

(Signature):

ACKNOWLEDGMENT

I am proudly grateful to my supervisors, Dr. R. Siyanbalapitiya and Mr. P. M. P. C. Gunathilaka, for showing me the correct path and having answers to all the questions I brought them, and for the advice which enabled me to follow on this research enthusiastically. I want to express my sincere gratitude towards all the Department of Statistics and Computer Science members.

I would like to thank the Dean of the Faculty of Science, Prof. S. R. Kodituwakku, Dr. A. J. Pinidiyaarachchi, the head of the department, and all the academic staff of the department for giving me the opportunity and guidance to continue working on this research.

My gratitude goes to my parents and sister for their continued support and encouragement, which made this research project exciting and successful.

ABSTRACT

Molecular pathway analysis has become significant and gained greater value in bioinformatics due to improvements in computational hardware. Molecular pathways can be defined as a network of molecules that take place within a cell. The reactions between molecules are important as the results directly relate to health. Pathways can mathematically be represented as graphs, and analysis can be performed on graph models. However, the number of pathway maps is substantial. Thus, with this method, Molecular Pathway Search (MPS) analysis runs for several days or even weeks, depending on the computation capacity of the machines. Accuracy against the execution time is considered the main issue in such scenarios. Researchers have yet to address this problem completely with limited computational power. Most previously developed graph-related algorithms are executed sequentially, and sequential algorithms do not fit well with high-performance computing. One of the main objectives of this work was to enhance MPS with the help of parallel computing techniques. The Depth First Search (DFS) algorithm was implemented. MPS is divided into individual search tasks and performs DFS on each task in parallel. When there are many nodes (molecules) and connections (interactions) in the graph, the traditional DFS techniques need a very high computational power. We introduced a new termination condition to overcome this problem in this work. The termination condition was controlled with a special parameter which controls the number of visits for a node depending upon the required level of accuracy and the available computational power. With the introduced parameter, all the previously uncompleted executions were completed. A cluster computing system with a 1TB shared memory was used to evaluate the proposed method, and depending on the termination condition, it has reduced the computation time significantly.

Key Words: Parallel Computing, Molecular Pathways

TABLE OF CONTENTS

DECLARATION	2
ACKNOWLEDGMENT	3
ABSTRACT	4
TABLE OF CONTENTS	5
List of Figures	7
List of Tables.....	8
CHAPTER 1	10
1 INTRODUCTION.....	10
CHAPTER 2.....	12
2 BACKGROUND.....	12
CHAPTER 3.....	16
3 METHODOLOGY.....	16
3.1 Data Set	17
3.2 File Processing.....	17
3.2.1 Parse XML File Using XPath	18
3.2.2 Species	18
3.2.3 Reactions, Modifiers and Product	19
3.3 Graph Building.....	20
3.3.1 Connect Graphs.....	21
3.4 Search.....	21
3.4.1 Depth First Search Implementation.....	21

3.4.2	Exclude and Include Option with Depth First Search.....	22
3.4.3	Solution for the non-termination problem in Depth First Search	23
3.5	Parallel Processing.....	23
3.5.1	Method 1	24
3.5.2	Method 2	25
CHAPTER 4		27
4	RESULTS AND DISCUSSION	27
4.1	Sequential Search	27
4.2	Parallel Processing.....	29
4.2.1	Parallel Search Method 1	31
4.2.2	Parallel Search Method 2	32
CHAPTER 5		34
5	CONCLUSION	34
CHAPTER 6		35
6	REFERENCES.....	35
CHAPTER 7		36
7	APPENDICES.....	36

List of Figures

Figure 2.1: Interrelationship among signaling pathways.....	14
Figure 3.1: processes	16
Figure 3.2: XPath Process	18
Figure 3.3: Species	18
Figure 3.4: Interaction between reactant, modifier and product.....	19
Figure 3.5: Reaction	19
Figure 3.6: Graph with two files.....	24
Figure 3.7: Method 2.....	26
Figure 4.1: Architecture.....	29

List of Tables

Table 4.1: Data Sets	27
Table 4.2: Result of Test_500 Data Set	28
Table 4.3: Results of method 1 with Test_700 Data set	31
Table 4.4: Results of method 1 with Test_1050 Data Set	32
Table 4.5: Cluster Results	32
Table 4.6: Time Comparison	33

List of abbreviations

DFS: Depth First Search, 25

DNA: Deoxyribonucleic Acid, 14

KEGG: Kyoto Encyclopedia of Genes and Genomes, 17

mRNA: Messenger RiboNucleic Acid, 16

SBML: Systems Biology Markup Language, 19

CHAPTER 1

1 INTRODUCTION

Analysis of the molecular pathways has gained significant practical value for researchers in various fields such as drug discovery, drug design, metabolic engineering for bioprocesses, human disease analysis, antimicrobial agents and therapeutics. The pathway can be defined as a network of molecules performing a cellular function. Pathway products depend on the requirements of the cell; they can be functional or structural. The pathways are essential for the growth and survival of the organism. Failure of a pathway to perform a cell function can lead to major diseases. So, pathways are critical in identifying diseases and drug discovery. Researchers and drug developers are more demanding of drug safety than ever before. The goal is to identify a molecule with the desired effect in the human body and to establish it quickly, safely and efficiently for treating patients. The latter requirements ensure that the approved medication improves patients' quality of life, not only by curing their illness, but also by ensuring that the cure does not cause other problems, namely, side effects. Effective drugs should always be considered for their toxicity. There are several types of pathways, such as signalling, genetic and metabolic. Metabolic pathways generate essential compounds such as amino acids, lipids, sugars, building proteins and cellular structures. Genetic pathways are involved in the regulation of genes. In the graphical representation of genetic pathways, nodes represent genes and edges represent the direct impact of genes on other genes. Determining the relationship between genes is very important for understanding gene regulation. Signalling pathways control the life processes, maintaining the equilibrium level for an organism to grow and survive. Analysis of signalling works helps in understanding biological systems and related diseases.

The quality of available information on molecular pathways for different organisms is rapidly increasing. It has now become possible to perform detailed analyses of metabolic pathway structure for the entire organism. For analysis purposes, pathways can be illustrated using 2-D diagrams whose nodes correspond to different molecules and edges represent the nature of molecule-molecule interaction. Pathway analysis can solve different types of problems faced by researchers. Such as

- Discovering all possible paths from compound A to compound B
- Calculating the number of paths and the steps within each path that lead from A to B
- Calculating the average distance between two compounds
- Identifying the distribution of path lengths between two compounds, the shortest path between two compounds
- Find all genes whose expression is directly or indirectly affected by a given compound
- Which pathway may be affected when one or more genes/proteins are turned off or missing
- Compare biochemical pathways from different organisms and tissues

However, pathway analysis is a challenging problem in systems biology and bioinformatics. The existing database is heterogeneous, and data can be incomplete, inconsistent, or approximate. The size of the pathway is huge. Since the number of pathway maps is substantial, the speed of pathway search is an issue. According to previous work, a comprehensive search takes weeks. For these reasons, an efficient program must proceed with and complete the search within a reasonable time.

CHAPTER 2

2 BACKGROUND

Over the last few decades, development in genomic and other molecular research technologies and information technology has led to a large amount of data related to microbiology. Processing this data manually is impossible. A new field called bioinformatics has been formed by combining biology with information science. Here, researchers can store and access data easily. Bioinformatics is an evolutionary field that develops methods and software tools for understanding large amounts of biological data. It combined computer science, statistics, mathematics and engineering to analyse and interpret biological data. It plays crucial roles in genetics and genomics and helps analyse biological pathways [2] in systems biology. The primary goal of bioinformatics is to enhance the understanding of biological processes. It uses the most computationally intensive computer science and statistics techniques, such as pattern recognition, data mining, machine learning algorithms and computer vision techniques. Major research areas in this field include protein structure prediction, gene finding, drug design, drug discovery and prediction of gene expression.

Proteins are long polymers of amino acids. A protein consists of one hundred to one thousand amino acids. Proteins differ from one another primarily in their sequence of amino acids. For the time being, twenty different types of amino acids exist. So, several different protein sequences are exponential. However, 15,000 protein structures are currently known. Proteins are essential to the life of organisms. They perform various functions within organisms, including DNA replication, transporting molecules from one location to another, participating in the immune system, and catalysing metabolic reactions.

The main function of protein is to bind to different substances. The protein's shape and the distribution of electrical charges on its surface decide what substance it will bind to. The shape of the protein depends on the amino acids it consists of. Many proteins are involved in the process of cell signalling and signal transduction.

These are intercellular processes. Some proteins, like insulin, transmit signals from one cell to another. Other membrane proteins act as receptors, responsible for binding signal molecules and passing biochemical responses into the cell. These receptors are located on the surface of the cell.

A gene describes what amino acids to assemble to build a protein. Genes are part of DNA molecules, large chains of nucleotides. There are four kinds of nucleotides: Adenine, guanine, cytosine and thymine. A codon combines three nucleotides that code for an amino acid. Some codons do not code for an amino acid but for the beginning or end of a protein being coded.

A molecule is an electrically neutral group of two or more atoms held together by chemical bonds. Biological pathways represent interactions between molecules within the cell under different conditions that lead to a biological reaction. Most of the time, these interactions are protein-protein interactions. Protein interactions assemble the signal cascades that transfer biological function and information through the pathway, and the proteins within the cell react with the environment by transducing signals. Signals from the external environment can trigger the activation of one or more biological pathways, which tackle specific tasks. For example, on or off genes spur cells to move. Different types of pathways exist in cells. They work together to perform tasks. Minor faults in these pathways can lead to significant diseases. Signalling, gene regulatory, and metabolic pathways are the central pathways in a cell. Signalling pathways process the chemical activities sent from outside the cell to the internal receptors. The Gene regulatory network represents transcriptional regulation activities between genes and transcription factors. Metabolites in a metabolic pathway interact inside the cell, with enzymes catalysing the metabolic reactions. Different chemical signals stimulate the proteins inside the signalling pathways to trigger specific reactions.

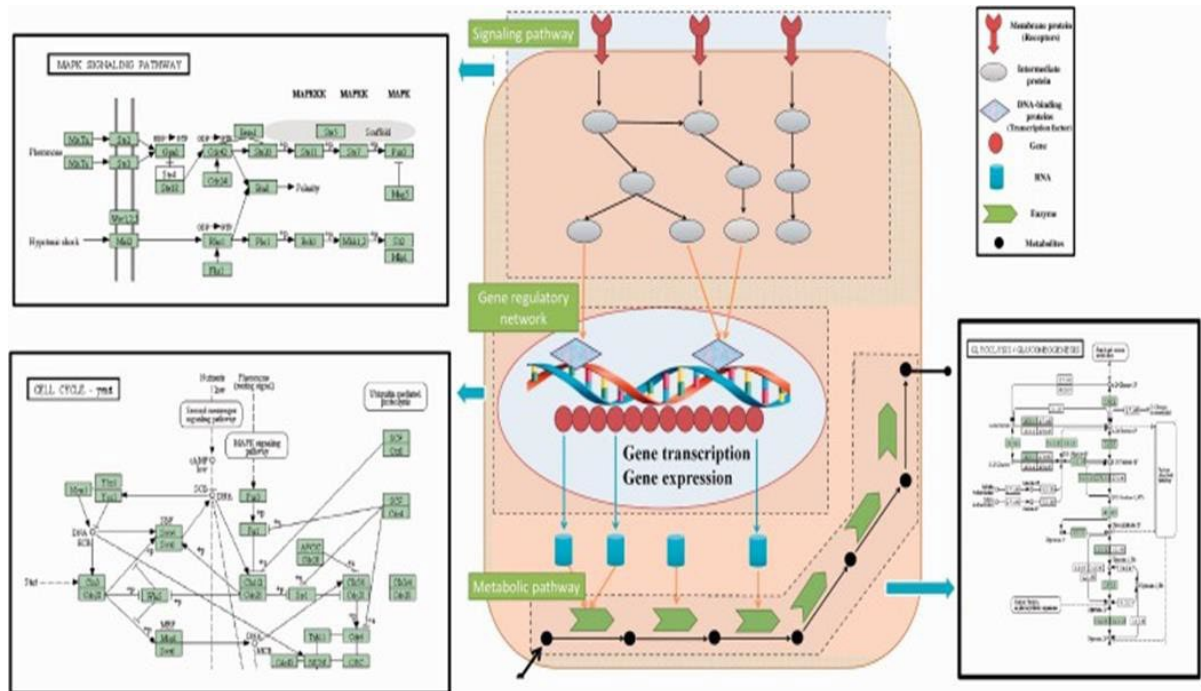


Figure 2.1: Interrelationship among signalling pathways [1]

The cell starts to recognise and receive the signals by activating membrane receptors, responding to the stimulus of the changes in the outside environment. Most of the receptors are proteins. Receptors transmit a signal from outside the cell into the signalling pathway inside the cell. Then a series of biochemical reactions begins. For example, take the MAPK signalling pathway in yeast (Top right). The stimuli of peptide mating pheromones, MatAalpha and Matata, initiate the signalling pathway. Pheromones are chemicals that can change the behaviour of the receiving individual. Then Ste2 and Ste3 receptors receive the peptide mating pheromones and activate MAPK signal transduction cascades. Signal cascade means a chain of biochemical events along a signalling pathway. The signalling cascade will finally produce the protein kinases that can enter the nucleus and activate the gene transcription by binding transcription factors to the promoters of DNA. In the example of the cell cycle yeast pathway (Bottom-left), the Cln3–Cdc28 protein kinases activate the transcription SBF and MNF, which regulate the Cln1/2 gene expression. The gene regulatory network can control the gene expression levels of mrna by activating the transcriptional factors and further translate mrna into protein. Specific proteins called enzymes participate in metabolic pathways. Enzymes catalyse chemical reactions. The molecules upon which enzymes may act are called substrates.

Enzymes convert substrates into different molecules known as products. Almost all metabolic pathways need enzymes to keep the reactions moving quickly enough.

Identifying the relationship between proteins, molecules, and genes in biological pathways will help us understand complex biological activities and functions. Most of the time, products from the cell depend on proteins and receptors. Different types of protein and receptor combinations can activate various types of pathways and finally end up with varying products of a cell. Identifying all these possible paths is very important for researchers and drug developers. Before deciding, they need to consider all the possibilities. In the case of drug development, some drugs may activate unwanted pathways. So, before making the drug, developers must analyse all paths.

Before analysing biological pathways, researchers need to store biological data properly. Researchers use databases to store the data. Pathway databases hold data on biochemical pathways, their components (enzymes, substrates, products), and their interactions. There are several databases for enzymatic catalysis, protein-DNA interactions, metabolic pathways, protein-protein interactions, such as KEGG (genes, enzymes, metabolic reactions), EMP (enzymes, pathways), and DIP (protein-protein interactions). The quality of the data model is essential for extracting suitable information for analysing the networks. Data models aim to represent experimental data and store it in a database. The database system provides data storage, management and retrieval functionalities. Traditional database models like relational models are unsuitable for analysing biological networks; thus, specialised data models such as graph-based models and computational models facilitate the analysis of these networks. These data models are usually implemented as in-memory data structures, containing information extracted from a database.

A standard data format must exist since different databases can export their data in various formats. SBML is one of the standard formats for representing biochemical networks. The format is based on XML. SBML is developed in levels. The lower level contains only basic information, and the higher level includes features for more specific information. Currently, it has reached level 2. There are elements such as species, listOfCompartments, reaction, and speciesReferences. Species represent chemical substances. A reaction consists of reactants, products and modifiers.

CHAPTER 3

3 METHODOLOGY

This chapter will thoroughly explain how the proposed method is formulated and the underlying mechanism of each step.

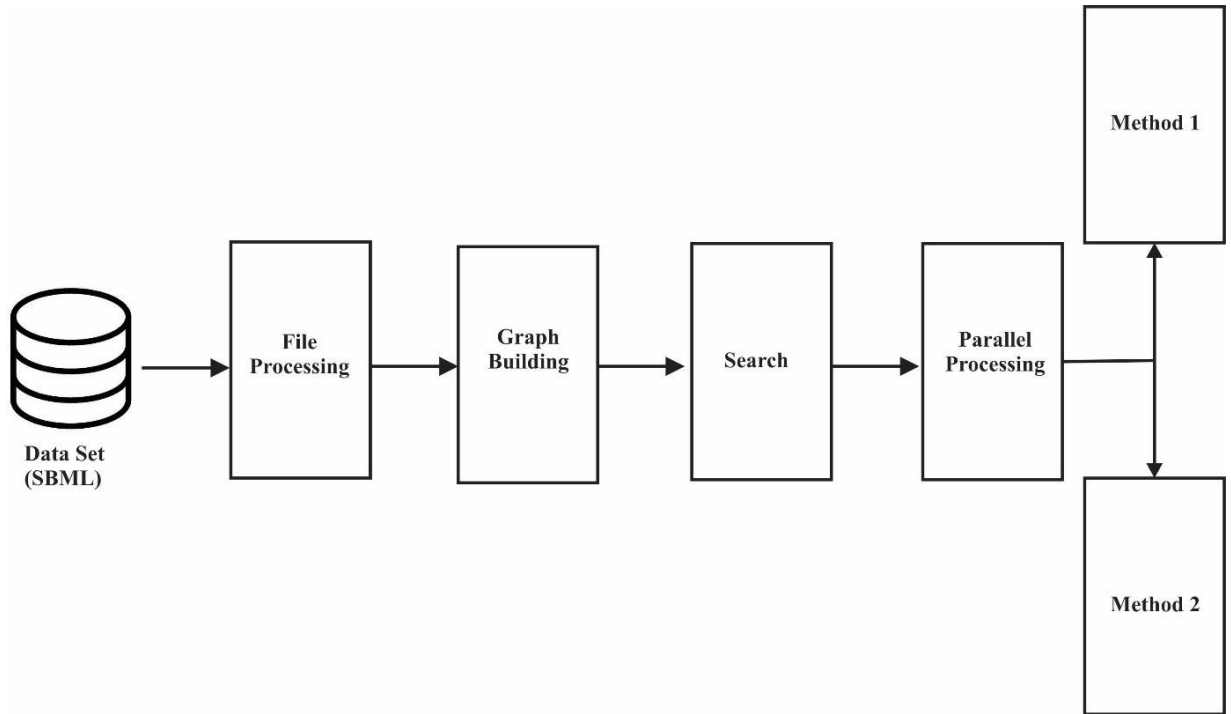


Figure 3.1: processes

3.1 Data Set

All collected data is in SBML file format, which is based on XML. Different representations of models are helpful for other purposes. Graphical diagrams of biological processes are helpful for visual presentation to humans. Still, at the software level, a different format is needed for quantifying a model to the point where it can be simulated and analysed.

That's where the Systems Biology Markup Language (SBML) comes in. SBML is a machine-readable format for representing models. It describes how biological entities interact with each other, such as a biochemical reaction. SBML can represent models commonly found in research, including cell signalling pathways, metabolic pathways, biochemical reactions, and gene regulation.

3.2 File Processing

Each SBML file is located on the hard disk. The main idea behind the file processing phase is to collect all the necessary information from SBML files and store it in appropriate data structures.

The file processing phase iterates through all the SBML files individually. Each SBML file corresponds to one graph. All these graphs need to connect to build the final graph, and all analysis processes are carried out. LinkedList and LinkedHashMap are the primary data structures. This information is needed to construct the graph. A strong language is required to find the information from the SBML file. In this method, XPath is used to navigate through SBML.

3.2.1 Parse XML File Using XPath

XPath is a language that finds information in an XML file. It is (sort of) SQL for XML files. XPath is used to navigate through elements and attributes in an XML document. The following figures show the steps of parsing an XML file with XPath.

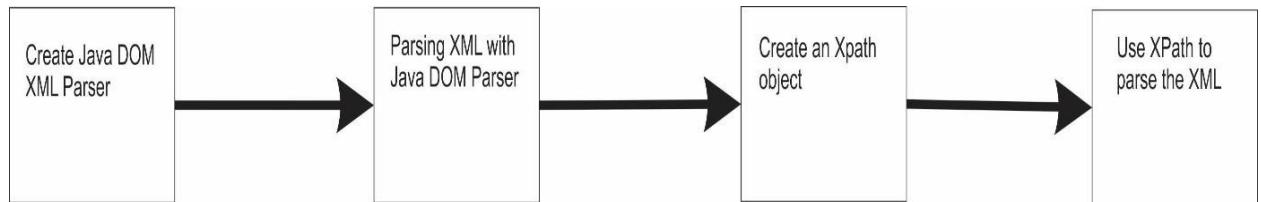


Figure 3.2: XPath Process

3.2.2 Species

Species ID and name are located in the listOfSpecies element.

```

<listOfSpecies>
  <species id = "C" name = "Cycline" compartment="cell">
  <species id = "K" name = "Active CDC-2 Kinase " compartment="cell">
</listOfSpecies>
  
```

Figure 3.3: Species

Here, we must store species ID and name attributes in species elements. HashMap is used as a data structure. The HashMap class implements the Map interface by using a hash table. It inherits the AbstractMap class and implements the Map interface. HashMap contains key-value pairs. It contains unique elements. HashMap maintains no order. Species ID is stored as a key, and species name is stored as a value. Species name concatenated with the file name. Assume the species name is A and its SBML file name is F1. The species name is represented as A@F1.

3.2.3 Reactions, Modifiers and Product

The list of reaction elements contains the reaction, modifiers, and product. Reactants and reactants lead to the creation of a product.

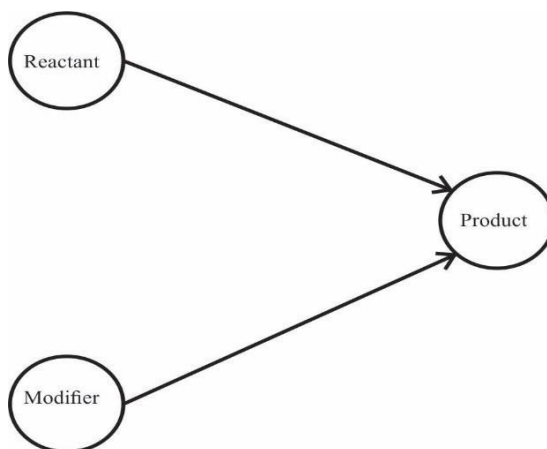


Figure 3.4: Interaction between reactant, modifier and product

Inside the reaction element are a list of reactants, a list of modifiers, and a list of product elements. The following figure shows how they are located.

```

<listOfReactions>
  <reaction id="r1" name="r1" metaid="_750280" sboTerm="SBO:0000177" reversible="true">
    <listOfReactants>
      <speciesReference species="X" />
      <speciesReference species="E" />
    </listOfReactants>

    <listOfProducts>
      <speciesReference species="P" />
    </listOfProducts>

    <listOfModifiers>
      <speciesReference species="S" />
    </listOfModifiers>
  </reaction>
</listOfReactions>
  
```

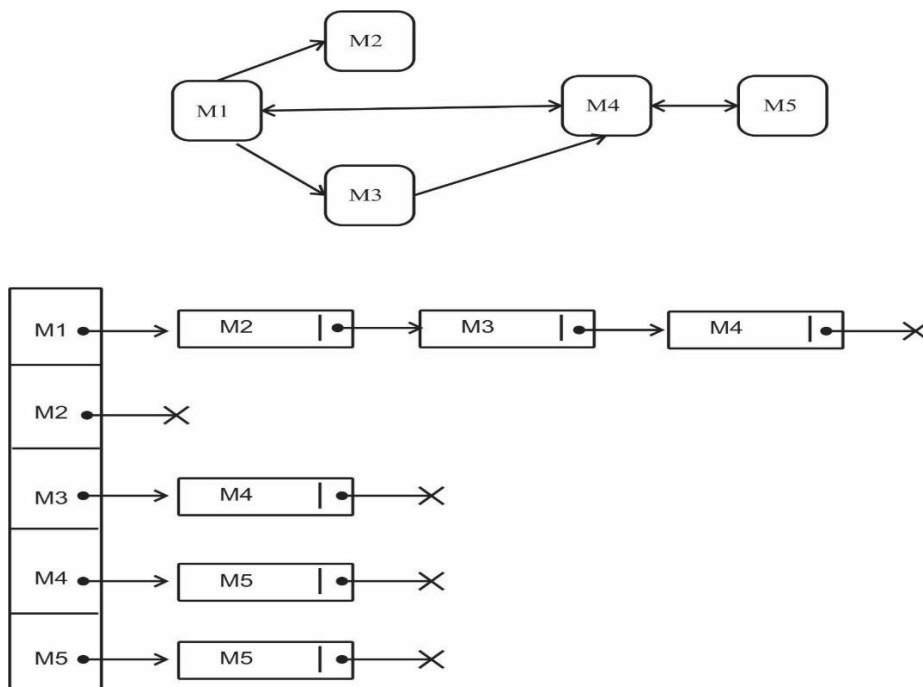
Figure 3.5: Reaction

The species attribute in the species Reference element represents the corresponding species ID for reactant, modifier, and product. The corresponding name for the species ID in the species Reference can be obtained using a previously stored species ID and species name.

The linked list data structure is used to store reactants and modifiers. LinkedList uses a doubly linked list to store the elements. It can contain duplicate elements and maintain insertion order. After storing the reactant, modifiers, and product separately, create edges. As shown in Figure 3.5, the edge is directed from reactant to product. There is another attribute called reversible in the listOfReaction element. If this attribute value is accurate, we must consider all products as reactions and reactions as products. A linked list is used as a data structure to store edges. Each LinkedList has two members corresponding to the reactant or modifier and the product. A list of LinkedList represents all edges.

3.3 Graph Building

After finishing the file processing part, the next step is the graph-building part. Edges have their directions, so the graph belongs to the directed graph. The HashMap data structure is used to build the graph. Key is a string, and value is a type of HashSet. Each key represents one node in a graph, and the value represents all its adjacent nodes.



3.3.1 Connect Graphs

After creating the graphs, each SBML file corresponds to one graph. These graphs need to connect. Assume M1 node in file 1 (M1@F1) and M1 node also in file 3 (M1@F3). So, there is a link between M1@F1 and M1@F3. M1@F1 and M1@F3 need to connect.

3.4 Search

A search is a set of steps for finding an item with specified properties among a collection of items. Different kinds of searching algorithms are used to perform different tasks. Each algorithm has advantages and disadvantages for performing the same task, depending on the data structure used, memory requirements, time complexity, etc. So, deciding which algorithm or data structure is most suitable for a specific task is difficult. The wrong choice of algorithm or data structure may lead to wrong results. So, when selecting a search algorithm, one has to consider many trade-offs. In this case, the search must sometimes go with a specific depth limit. Always start with the root node and need to go node by node until the end node is found or until no more adjacent nodes are found. The best suitable algorithm for this scenario is Depth First Search (DFS).

3.4.1 Depth First Search Implementation

Depth First Search [3] is a general technique used in artificial intelligence for solving a variety of problems in planning, decision making, theorem proving, expert systems, etc. DFS is an algorithm for searching or travelling in a graph or a tree. It's expanding the adjacent node of the root and then expanding the next adjacent node until it visits the destination node or one that has no more adjacent nodes. Then search backtracks to the last visited node.

I have used the recursive implementation of Depth First Search in this method. LinkedList is used as a data structure. Source and target nodes are given as the algorithm's inputs. I have used a LinkedList data structure called “visit” to keep track of the visited nodes from the source to the target. The source node is inserted into the visit, and all its adjacent nodes are checked twice using two loops.

In the first loop, check if the adjacent node equals the target node. If it does, it is inserted into the visit, and all nodes take the result path. After storing the result path, the last inserted node of the visit is removed. If the adjacent node is already on the visit, it will be skipped.

The second loop will be skipped if the adjacent node has already been visited or is equal to the target node. Otherwise, the adjoining node will be inserted into the visit, and the algorithm will run recursively.

3.4.2 Exclude and Include Option with Depth First Search

Excluding search means there is a set of predefined nodes, and in the result path, one more predefined node exists, so that the result path is not taken as a result. The excluded nodes are given as user input to the algorithm. Excluded nodes are stored in a LinkedList data structure called the exclude List. As mentioned in 4.1, this exclusion node condition must also be checked in the second loop. If the adjacent node equals one of the nodes in the exclusion List, it will skip.

Including search means there is a set of predefined nodes, and only paths with at least one of the predefined nodes exist in the path taken as a result. After finishing the search, all paths must be filtered to get the search results with the included option. As a result, all paths have at least one node.

3.4.3 Solution for the non-termination problem in Depth First Search

The algorithm mentioned in section 3.4.1 works fine for smaller graphs (fewer nodes). But this algorithm ends up in an infinite loop in the highly connected graphs. This section discusses how to avoid the problem of non-termination.

The main reason for this non-termination problem is that the DFS search repeatedly goes through the same set of nodes. The solution is to keep track of the number of times a node is visited in the algorithm. The use of node count can avoid this problem. We need to make some modifications to the algorithm discussed in section 3.4.1. A hash map data structure called “node count” must be created. In the second loop, we must keep track of the adjacent nodes. The key of the node count belongs to the adjoining node name, and its corresponding value represents the number of times it appears as the neighbouring node. It will be skipped if this node count value exceeds a predefined value.

3.5 Parallel Processing

After building the graph, the search process must be conducted. The sequential process of this method takes a considerable amount of time. Parallel computing is needed to reduce computing time. Parallel computing is a form in which many calculations are carried out simultaneously. It operates on larger problems that can be divided into sub-problems, which are then solved concurrently. In recent years, parallel computing has been widely adopted in domains that need massive computational power, such as graphics, animation, data mining and informatics. Parallel computing uses multiple computer resources simultaneously to solve complex problems.

This method can parallelise two phases: Graph building and search. The file processing phase can't be parallelised. Parallel search takes advantage of multiprocessors with shared memory to reduce computation time. The following section discusses how the search can be run in parallel. In this method, I have used two methods to decompose the search.

3.5.1 Method 1

The following Figure shows a graph with two files.

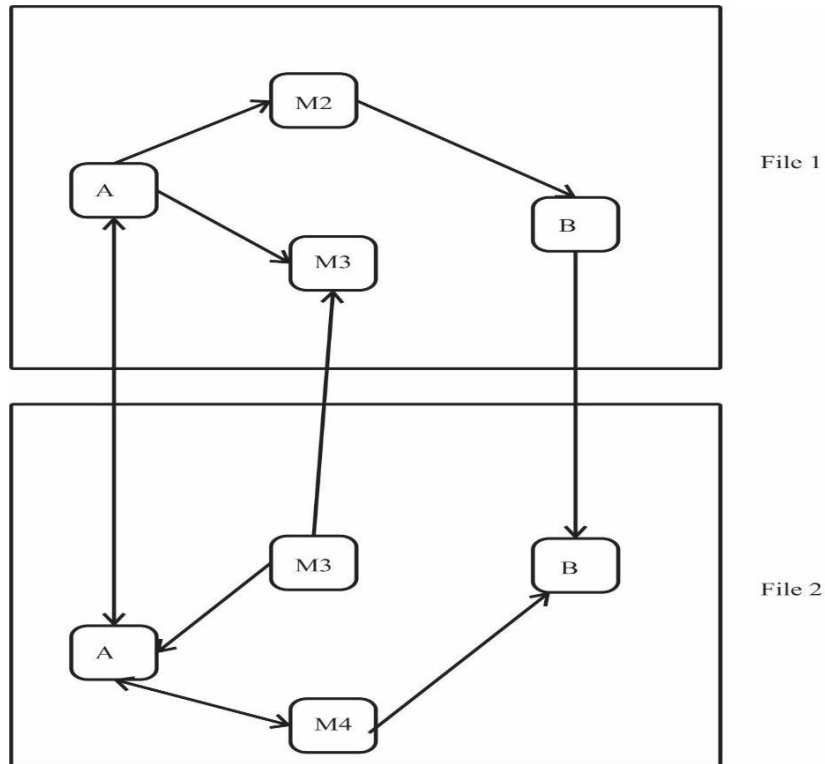


Figure 3.6: Graph with two files

The source node is A, and the target node is B. The Goal is to get all source nodes in the graph and store them in a List data structure. In this case, A@File_1 and A@File_2. Two different search tasks start from these two nodes. These two searching tasks can run concurrently as task 1 and task 2. We must create a thread pool using an executor service [4]. These two tasks are submitted to the thread pool as Java callable [6]. The graph must be cloned before submitting the functions to the thread pool. The thread pool executes the tasks asynchronously, giving the result as a future object [7]. So, we need to create an appropriate data structure to store the future objects

When two tasks are carried out concurrently, it may be possible to run the same search repeatedly in two threads. In Figure 3.6, the search starts from A@F1 and A@F2. Possible results are shown below.

Task 1:

A@F1 -> A@F2 -> M4@F2 -> . . .

Task 2:

A@F2 -> M4@F2 -> . . .

Tasks 1 and 2 run the exact search, which must be avoided. So, we must clone the graph and check the adjacent nodes from the source node to the first neighbouring node needing removal.

3.5.2 Method 2

This method is illustrated in the following diagram.

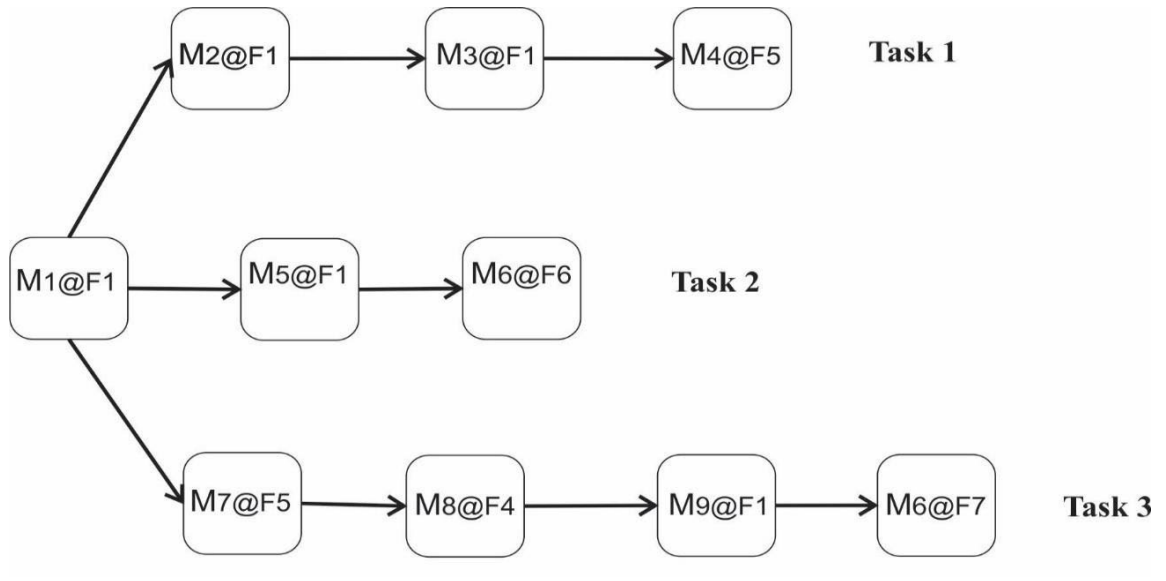


Figure 3.7: Method 2

CHAPTER 4

4 RESULTS AND DISCUSSION

This chapter will thoroughly go through the results of the methodology discussed in Chapter 3. Methods are implemented in the Java programming language to detect possible pathways in a biological network. Then, we evaluated the methods using the dataset with a cluster and a laptop pc. The cluster has 72 cores with 1TB of shared memory. The laptop has a 1.60GHz Intel(R) Core (TM) i5-4200 CPU and 6.00GB of memory. It's running on a Windows 10 64-bit operating system. The complete data set contains 1050 SBML files. However, for testing purposes, the dataset has been divided into different-sized datasets.

Table 4.1: Data Sets

Name of sub-data set	Number of SBML files
Test_1050	1050
Test_700	700
Test_500	500

4.1 Sequential Search

The main goal of molecular pathway searching is to find all possible pathways between the source and the destination. I have used Depth First Search as my searching algorithm in this thesis. The choice of the searching algorithm depends on the purpose of the search. Here, we must get all possible paths between two nodes in a graph. There are other requirements in the search. Such as a search with a depth limit, including search, and excluding search. Breadth First Search is also like Depth First Search. But in BFS, there is no way to limit the depth. Compared to BFS, DFS has a much lower memory requirement.

For the implementation of DFS, we have used a linked list data structure. ArrayList can also be used for this implementation. Unlike ArrayList, LinkedList has many benefits. Manipulation with ArrayList is slow because it internally uses an array. All the bits are shifted in memory if any element is removed from the array. However, manipulation with LinkedList is faster than ArrayList because it uses a doubly linked list, so no bit shifting is required in memory. ArrayList is only better for storing and accessing data. But LinkedList is better for manipulating data.

Traditional DFS search implementation starts from the root node, goes as deep as possible, and then backtracks. However, in that implementation, there is no way of keeping track of paths one by one. It only gives the nodes. In this algorithm, I have used another data structure to keep track of the paths visited one by one. So that it can keep track of all paths. The algorithm is capable of excluding and including search. Excluding search means that there is a set of predefined nodes, and the search is carried out without these nodes. The exclude search is happening in real time. Before DFS visits a node, it checks if it is on the exclusion list. If a node in the exclude list search is stopped, it backtracks to the last recently visited node. Including search means paths with predefined nodes. If they include a node not in the result path, that path is not taken. Include the search after the search finishes.

Sequential search is tested with all three datasets (Test_500, Test_700, Test_1050) on a laptop pc. Data set: Test_500

Start molecule: JAK2

Source molecule: stat5-p

Table 4.2: Result of Sequential Search with Test_500 Data Set

Max Visit	Number of Paths	Time
10	17	00.00.58
100	110	00.01.37
1000	1101	00.07.06
10000	9914	00.51.52

4.2 Parallel Processing

Sequential searches require more time to complete. Parallel processing techniques can reduce computation time considerably. The file processing phase cannot be done in parallel. This proposed method reads files from only one hard drive, so there is only one head for reading. Searching processes can be parallelised. The DFS search is also used as the search algorithm for parallel search.

Java can meet the needs of high-performance computing, where some other programming languages have failed. It's simple, object-oriented, portable, robust, and extensible; a thread-based execution model makes it a suitable candidate for a parallel platform. The Java course is compiled into a class file that contains machine-independent bytecode. This byte-code form can easily be transmitted over the network to a Java-enabled client, which is interpreted by the Java Virtual Machine (JVM).

The following figure shows a typical shared memory multicore architecture, in which the multicore processors share the same memory space. Each core is essentially a processor.

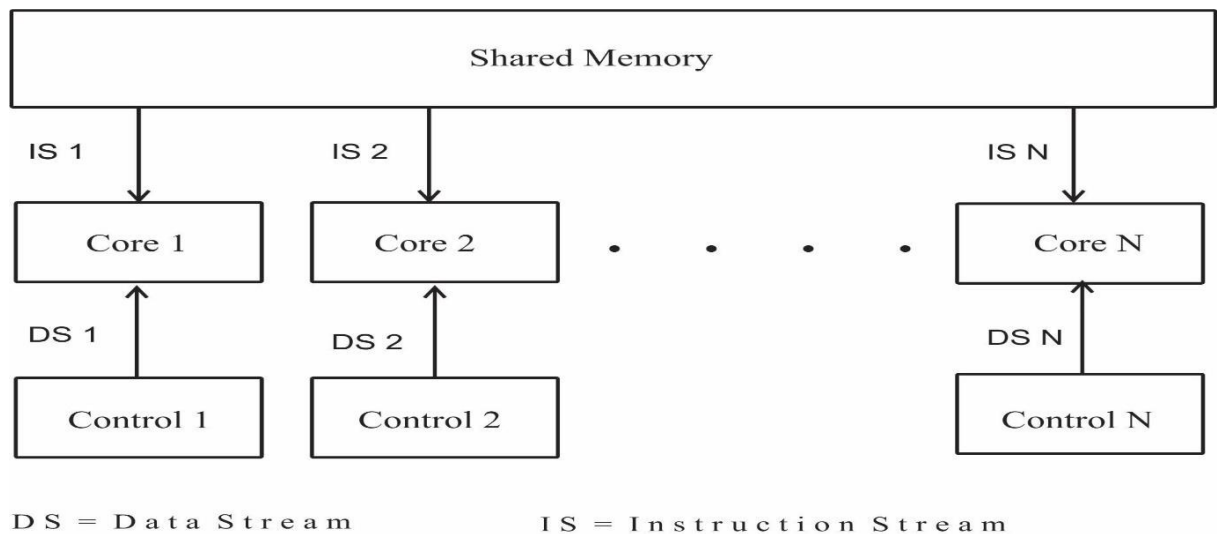


Figure 4.1: Architecture

Application designs for shared memory architecture need Java threads or Open Message Passing implementation (MPI) for archive parallelism. Due to the need for high-performance computing, Java threads for parallel programming are exceptionally extended. The use of threads in large applications requires significant and careful complexity and programming effort. To achieve thread programming, Java has provided a set of concurrent utilities such as thread pools, tasks, and executors.

Java threads are a natural choice for parallel programming as the thread class is a part of the standard Java libraries since the Java 1.0 specification. Threads are lightweight processes that exist within a process and share resources like memory, files, etc. Threads use fewer resources compared to their typical process creation. So that enables efficient communication. Traditionally, concurrent execution was achieved by a multi-threaded processor; on a single processor, process execution resources are allocated between threads. With recent JVMs (1.8), in a shared memory multi-threaded process, each thread can run on a separate processor simultaneously, resulting in parallel execution. This is because current JVMs implement threads on top of native OS threads, which allows them to be scheduled on different processes to achieve parallel execution. However, creating and terminating threads are expensive, so instead of creating a new thread for each parallel task, maintaining a fixed pool of threads for the entire program duration and queuing up parallel tasks often results in more efficiency.

Executors are a high-level service capable of running asynchronous tasks and typically manage a pool of threads, so we do not have to create new threads explicitly. They enable encapsulated access to thread pools, thus decoupling thread management and creation from the rest of the application. This greatly reduces the developer's effort to manage thread pools, synchronisation and scheduling problems, allowing more focus on the actual parallel computation problem.

Since an executor service manages a pool of threads reused under the hood, we can get away with as many concurrent tasks as needed for the application execution with a single executor service. Executors [4] class provides various factory methods for creating different executive services

Callable is a functional interface introduced in the Java 1.5 specification to complement the existing Runnable interface. Unlike Runnable, it is usually utilised to submit a task to a thread or a thread pool for asynchronous execution to either return a result or throw an exception. Callable represents an asynchronous computation whose value can be accessed via a Future object. All the code that must be executed asynchronously goes into the call method. When a Callable is passed to the thread pool, it chooses one thread and executes the Callable to return a Future object to hold the result of the computation once complete. The get () method of Future can then be used to return the calculation result or block if the computation is not finished. An overloaded get () method with a timeout is also available to avoid indefinite blocking. Future also allows us to cancel the task if it's not started or interrupt it if it's already started. Classes like Integer, String, etc., can be wrapped using Callable and Future.

4.2.1 Parallel Search Method 1

The parallel search method was run with different data sets on a laptop PC and a cluster. The results are given below.

Data set: Test_700

Source Molecule: JAK2

Target molecule: stat5-p

Table 4.3: Results of method 1 with Test_700 Data set

Maximum Number of Visits	Number of Links	Time
10	0	00.02.24
100	102	00.05.01
1000	1471	00.23.24

Data Set: Test_1050

Source Molecule: JAK2

Target molecule: stat5-p

Table 4.4: Results of method 1 with Test_1050 Data Set

Maximum Number of Visit	Number of Links found	Time
10	1	00.09.12
1,000	333	00.45.23
10,000	3377	1 days 13.56.47
100,000	32830	9 days 03.57.03

Table 4.5: Cluster Results

4.2.2 Parallel Search Method 2

Method 2 was tested with all datasets using a laptop pc. Results are given as

follows. Data set: Test_500

Source Molecule: JAK2

Target molecule: stat5-p

Table 4.6: Results of Method 2 with Test_500 data Set

Maximum number of visits	Number of Links found	Time
10	17	00.00.36
100	110	00.01.01
1000	1101	00.03.09
10000	9914	00.23.52

As we can see from the results for smaller graphs, there is no longer a time difference between sequential and parallel algorithms. Sometimes, parallel search may take longer than sequential search in smaller graphs because of the time it takes to get the JVM up in every core. However, parallel search takes less time in larger graphs than sequential search.

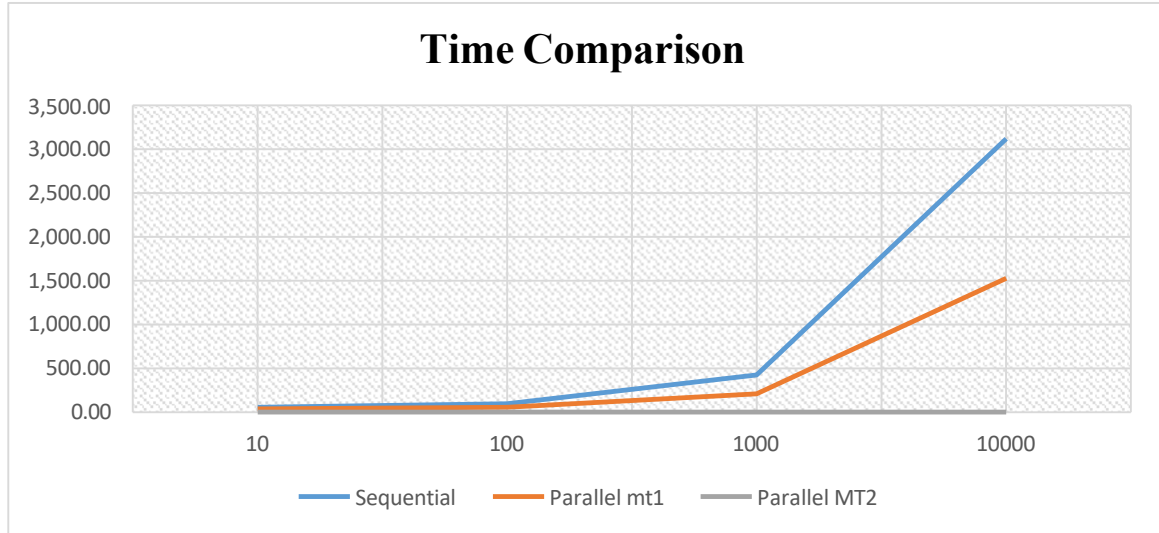


Table 4.7: Time Comparison

CHAPTER 5

5 CONCLUSION

Molecular pathway analysis is the most trending research field in computer science and bioinformatics. Researchers and developers make predictions and act based on the results of the path analysis. However, due to the substantial number of pathway maps, they lack an effective pathway search. This thesis provides an effective and efficient molecular pathway analysis method. Previous analysis methods take 3 to 4 weeks to complete. However, this paper shows that time can be reduced to 10 days. A person who runs this program needs basic knowledge about molecular pathways. Results depend on the user's inputs. The program is implemented as a bundle. The user can input all the information needed into the program using a property file.

CHAPTER 6

6 REFERENCES

- [1] Jie Hou, Lipi Acharya, Dongxiao Zhu and Jianlin Cheng (2016). “An overview of bioinformatics methods for modeling biological pathways in yeast”
- [2] Yves Deville, David Gilbert, Jacques van Helden and Shoshana J. Wodak (2003). “An overview of data models for the analysis of biochemical pathways”
- [3] Navneet kaur, Deepak Garg (2012). “Analysis of The Depth First Search Algorithms”
- [4] Oracle Java Specification Documentation Executors.Available:
<https://docs.oracle.com/javase/tutorial/essential/concurrency/executors.html>
- [5] Oracle Java Specification Documentation: Interface ExecutorService.Available:
<https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/ExecutorService.html>
- [6] Oracle Java Specification Documentation: Callable. Available:
<https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/Callable.html>
- [7] Oracle Java Specification Documentation: Future. Available:
<https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/Future.html>

CHAPTER 7

7 APPENDICES

Config.property file

```
Start_molecule = JAK2
End_molecule = STAT5-P
exclude = basal@stat5
include =
Depth_limit = 5
File_path = E:/Test_500/
node_count_value = 1000
number_of_cores = 4
```